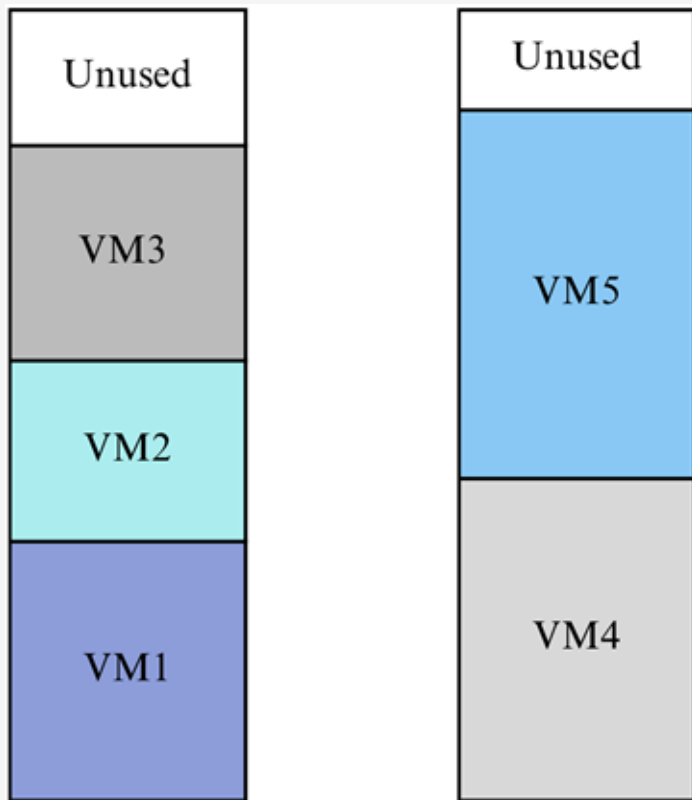


Aggregate VM: Why Reduce or Evict VM's Resources When You Can Borrow Them From Other Nodes?



Deniz Can Gezgin 27992
Sueda Şeker 28325
Oytun Kuday Duran 28357

Hardware resource fragmentation in data centers



Resource Fragmentation: Presence of small segments of resources that are too small to be utilized.

Fragmentation level of a resource is an indicator of the quality of resource utilization.

Even considering optimized placement methods, or attempts to defragment resources through migration, resource fragmentation still persists.

What exactly are we trying to solve?

We can define a mapping *size* from set of each virtual machines (VMs) and physical machines (PMs) to non-negative numbers as follows:

Let P be a set of PMs

$$\text{size}_{\text{PM}} : PM \in P \mapsto \mathbb{N}_0$$

Let V be a set of VMs

$$\text{size}_{\text{VM}} : VM \in V \mapsto \mathbb{N}_0$$

We consider a distributed cloud environment C as a finite set of physical machines (PMs):

$$C = \{PM_1, PM_2, PM_3, \dots, PM_n\}.$$

where each PM is a finite set of virtual machines (VMs):

$$PM_i = \{VM_1^i, VM_2^i, VM_3^i, \dots, VM_n^i\}.$$

And for any $PM_i \in C$:

$$\text{size}_{\text{PM}_i} \geq \sum_{VM^i \in PM_i} \text{size}_{\text{VM}^i}$$

What exactly are we trying to solve?

Our problem is to place n VMs on available PMs to minimize the fragmentation index:

$$\text{Fragmentation Index} = \frac{\text{Aggregate free capacity of PMs which cannot fit a VM}}{\text{Total available free capacity}}.$$

More formally, define free capacity $F : C \rightarrow \mathbb{N}_0$ as follows:

$$F(PM) = \text{size}_{PM} - \sum_{VM \in PM} \text{size}_{VM}$$

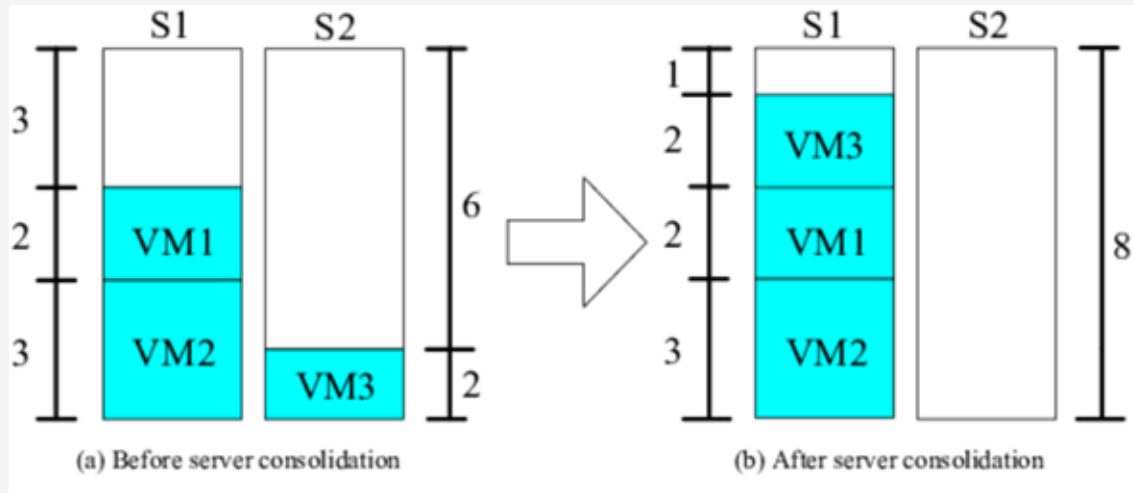
Given a minimum size needed to fit a virtual machine S , the set of all PMs that cannot fit a VM can be defined as follows:

$$X = \{PM \in C \mid F(PM) < S\}.$$

Now, we can formally define the fragmentation index as follows:

$$\frac{\sum_{PM \in X} F(PM)}{\sum_{PM \in C} F(PM)}$$

Problems with current solutions - Consolidation



[3]

Ideal VM placement solves the resource fragmentation problem.

But it is NP-Hard (from reduction to 0-1 Knapsack).
[1][2]

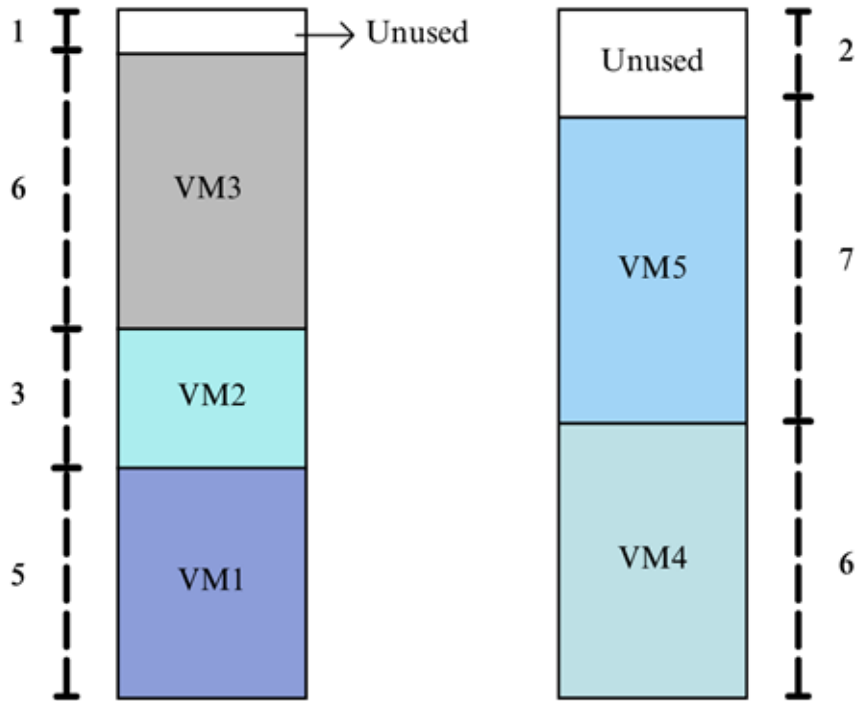
Data center operators are looking for solutions

Microsoft: On Azure's scale, even 1% in fragmentation reduction can lead to cost savings in the order of \$100M per year

Facebook and Bing: Fragmentation and over provisioning are responsible for a 45% increase in job makespan (total length of the schedule).

Eolas cloud provider: Average of 17% of the physical resources wasted each day due to fragmentation.

Problems with current solutions - SpotVM



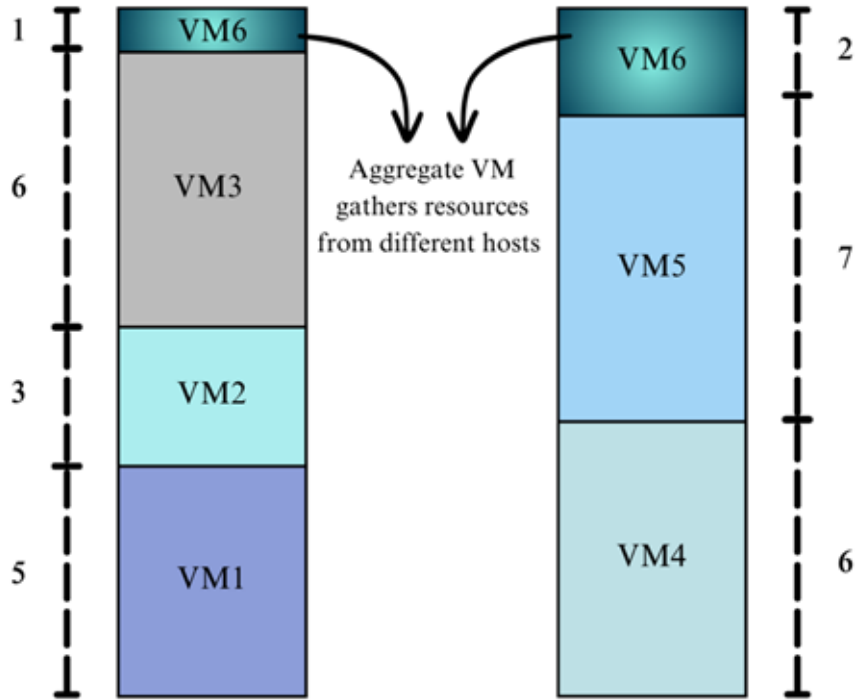
The solution of SpotVM is to monetize fragmented resources, but there are several problems such as

- Only a certain minimum amount of resources
- Still a need of finding a certain amount of resources
- Might be evicted at any time

Thus, not adequate for Primary VMs' workloads.

Hence, there is still a search for VMs with guaranteed resources.

A different approach to the solution: Aggregate VM



Solution: Rather than making use of available resources on a single machine, combining scattered resources from multiple machines to create a single distributed virtual machine.

Guarantee of certain fixed number of resources at all times, without eviction.

However, performance is based on the type of the workload.

A different approach to the solution: Aggregate VM

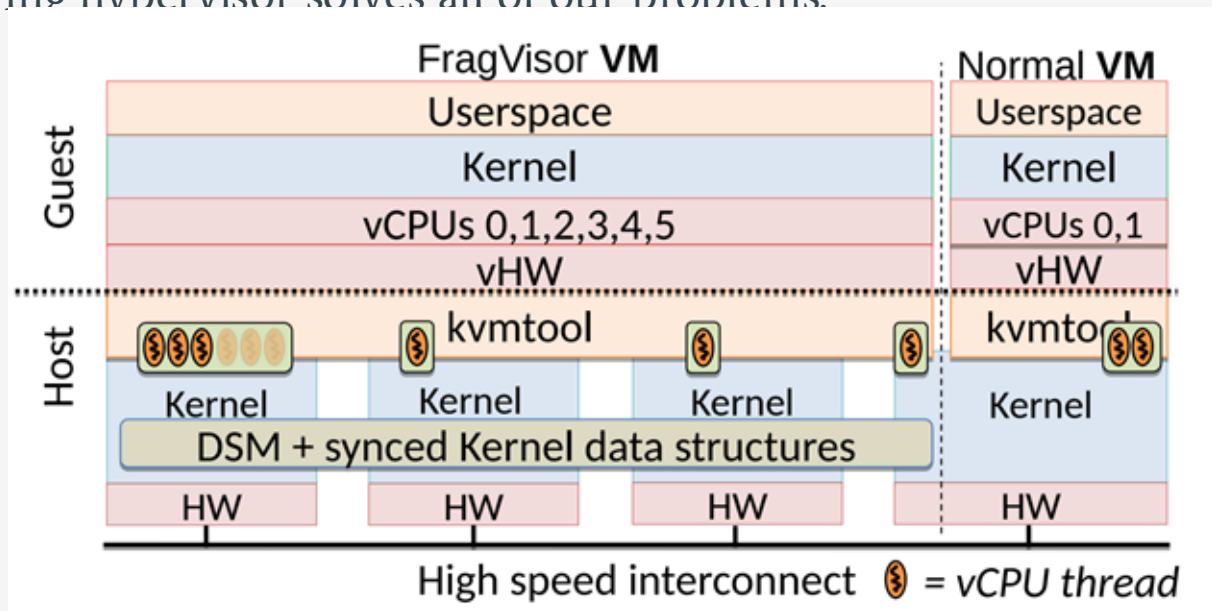
Therefore, this paper is driven by the following questions:

- a) Can Aggregate VMs be a solution to the fragmentation problem, and at what overhead?
- b) Can we achieve guaranteed resources provisioning (no reduced resources, nor evictions contrary to SpotVM)?
- c) For what type of workloads would be the best fit?

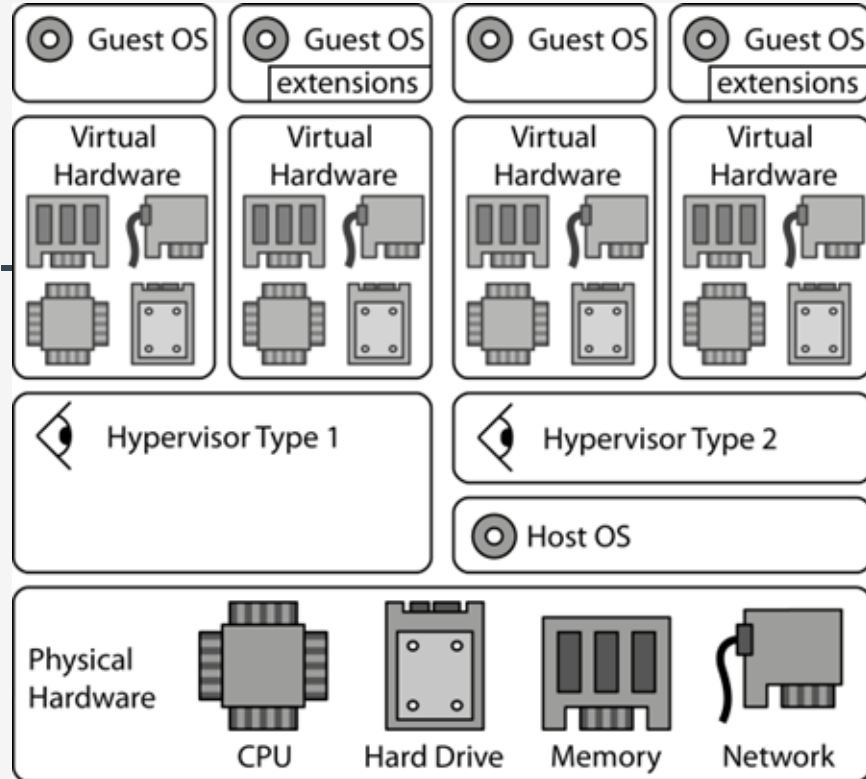
Claim

A type 2 resource-borrowing hypervisor solves all of our problems.

- What?
- How?



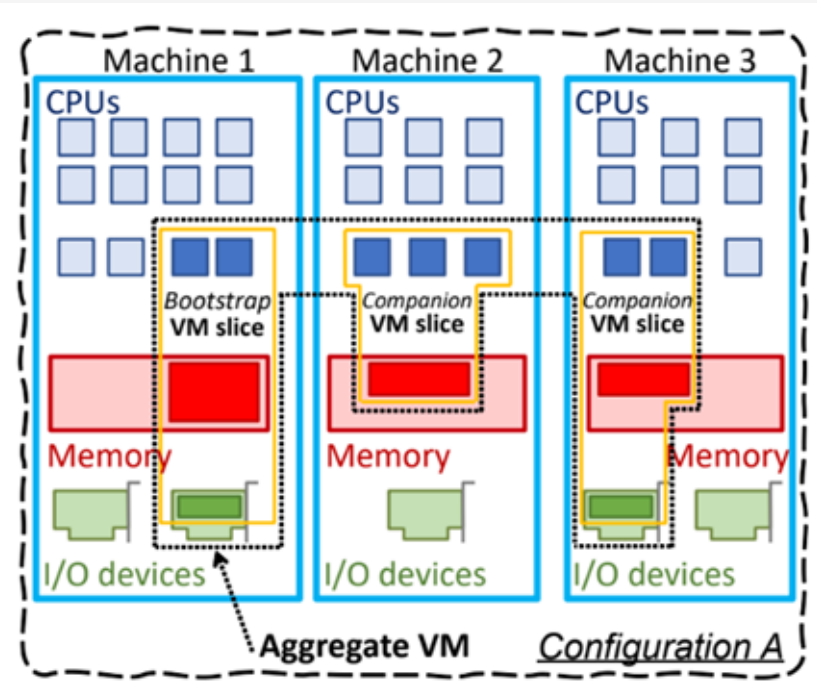
Distributed Resource Borrowing Hypervisor



A **Hypervisor** creates virtual hardware from physical hardware

- Type-1 directly runs on the hardware,
- Type-2 coexists with or is an OS service, e.g. KVM
- With type 2, virtualized CPUs may run as host OS threads

Distributed Resource Borrowing Hypervisor

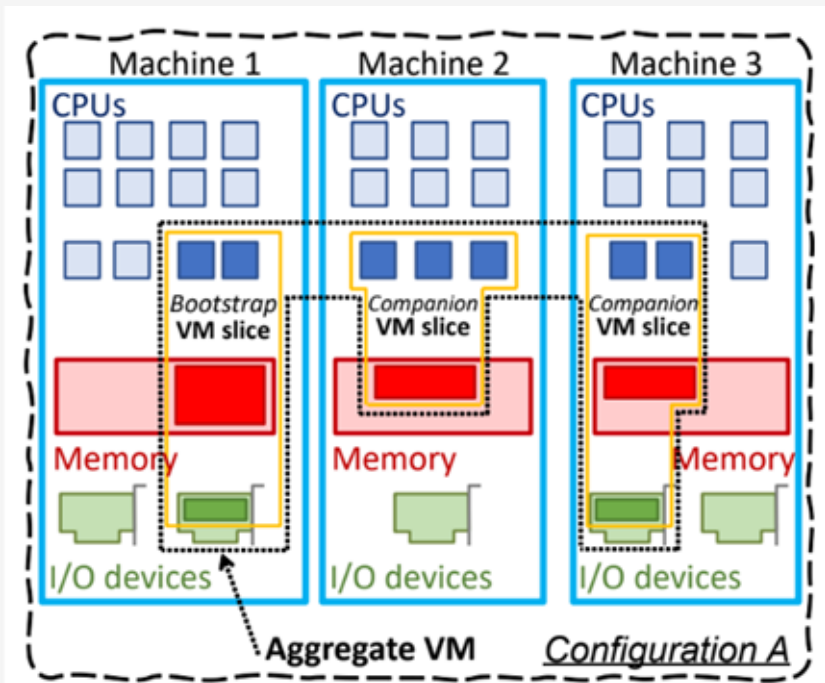


Hypervisor instances run on different servers
= **distributed**

Instances talk with each other using a
communication layer (**message-passing**)

This creates an illusion of a single VM among
multiple hypervisors = the Aggregate VM

Distributed Resource Borrowing Hypervisor



A hypervisor design that **provisionally** aggregates “slices” of fragmented hardware

Borrowing demands for mobility:

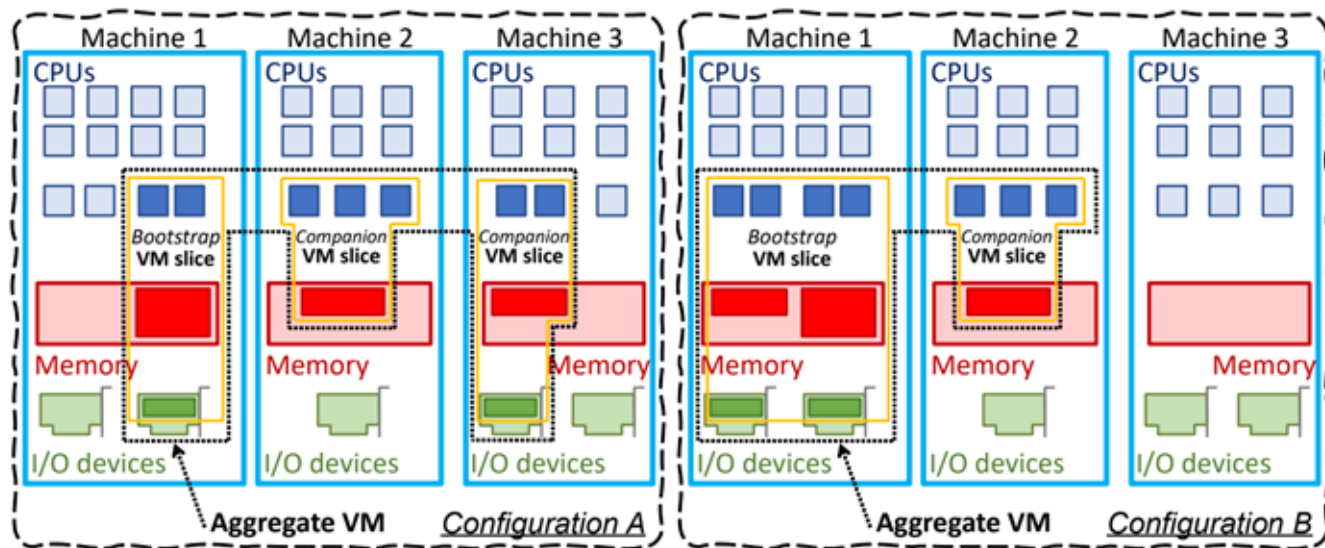
- **vRAM mobility**
- **vCPU mobility**
- **I/O device mobility**

Dynamically reconfigurable to leverage resources on different nodes

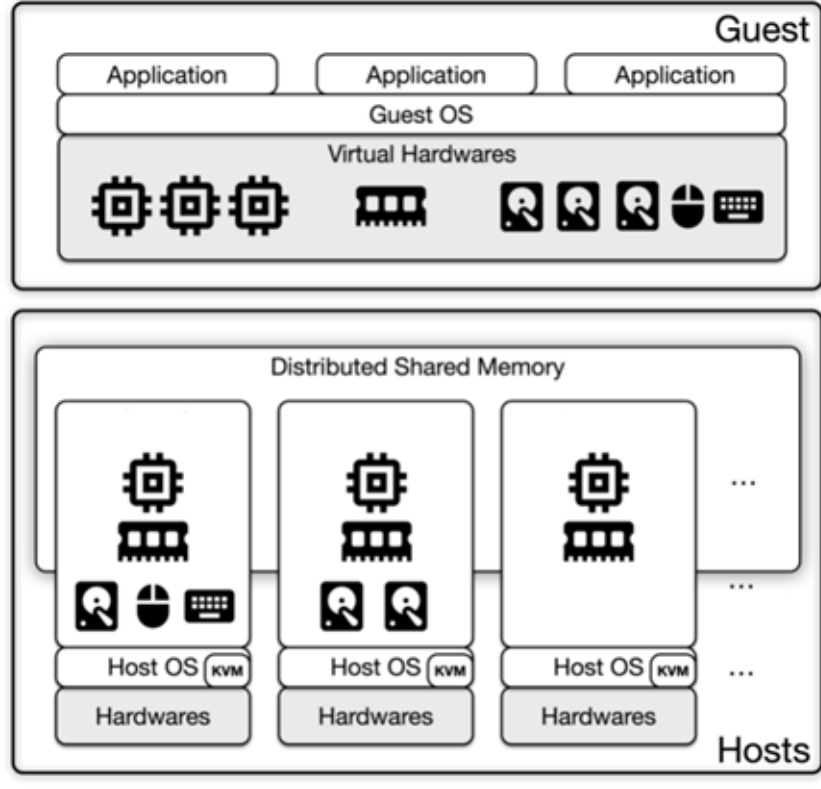
Dynamically reconfigurable: A > B

VM can transparently move at runtime when hardware resources become available (not possible with existing distributed hypervisors)

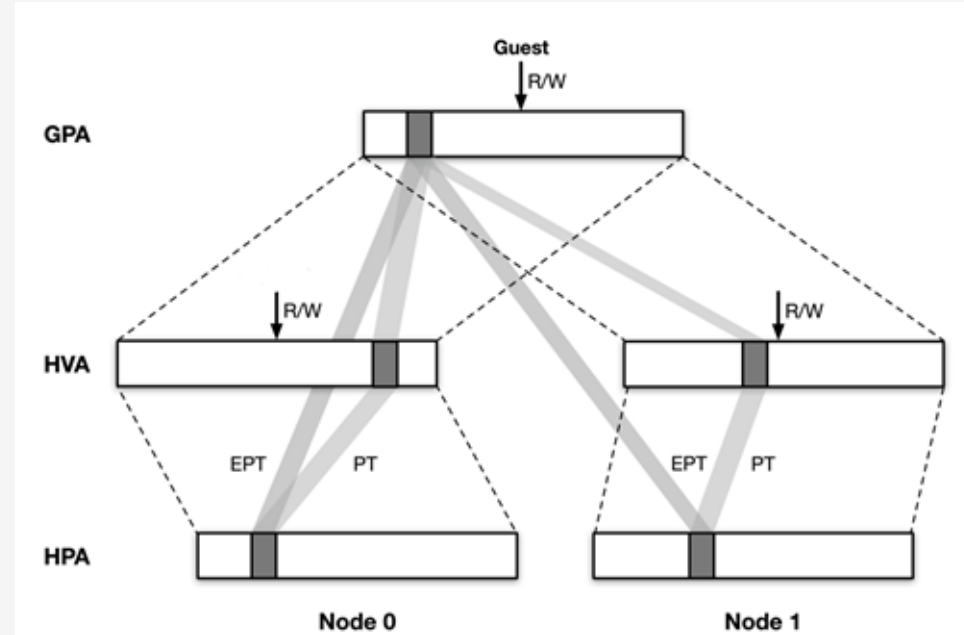
- Don't have to evict like SpotVM
- This allows us to consolidate a VM over time on as few servers as possible – ideally a one



Distributed Shared Memory (DSM)

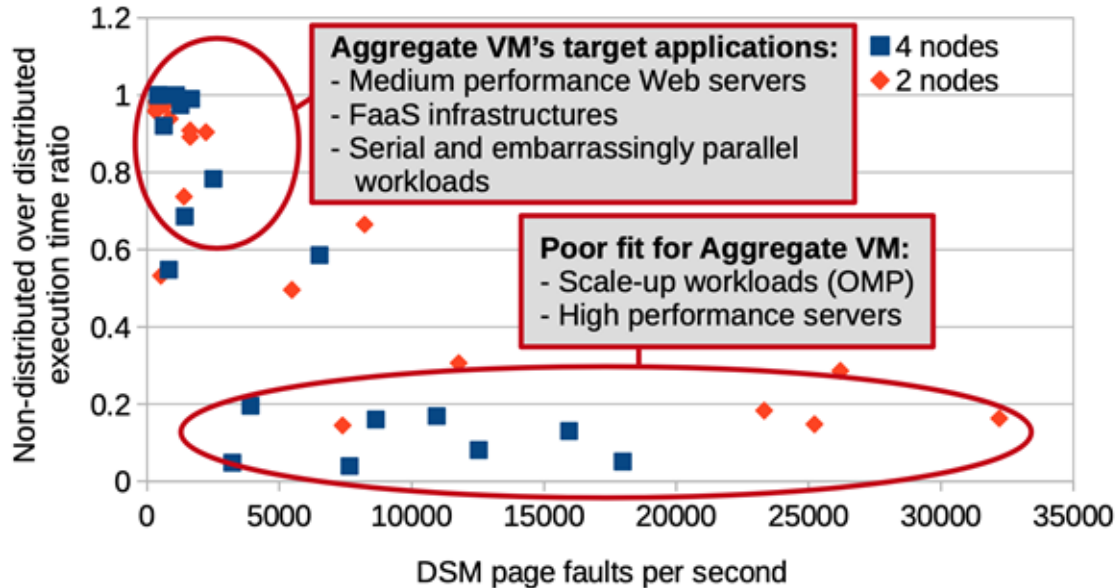


- Hosts allocate guest RAM in their memory.
- Guest sees one coherent address space



Which workloads are less affected by DSM?

Page faults are known to be slow, huge overhead to keep DSM consistent



FragVisor

FragVisor, a type 2 resource-borrowing hypervisor based on Linux/KVM that is built to create and maintain the Aggregate VMs

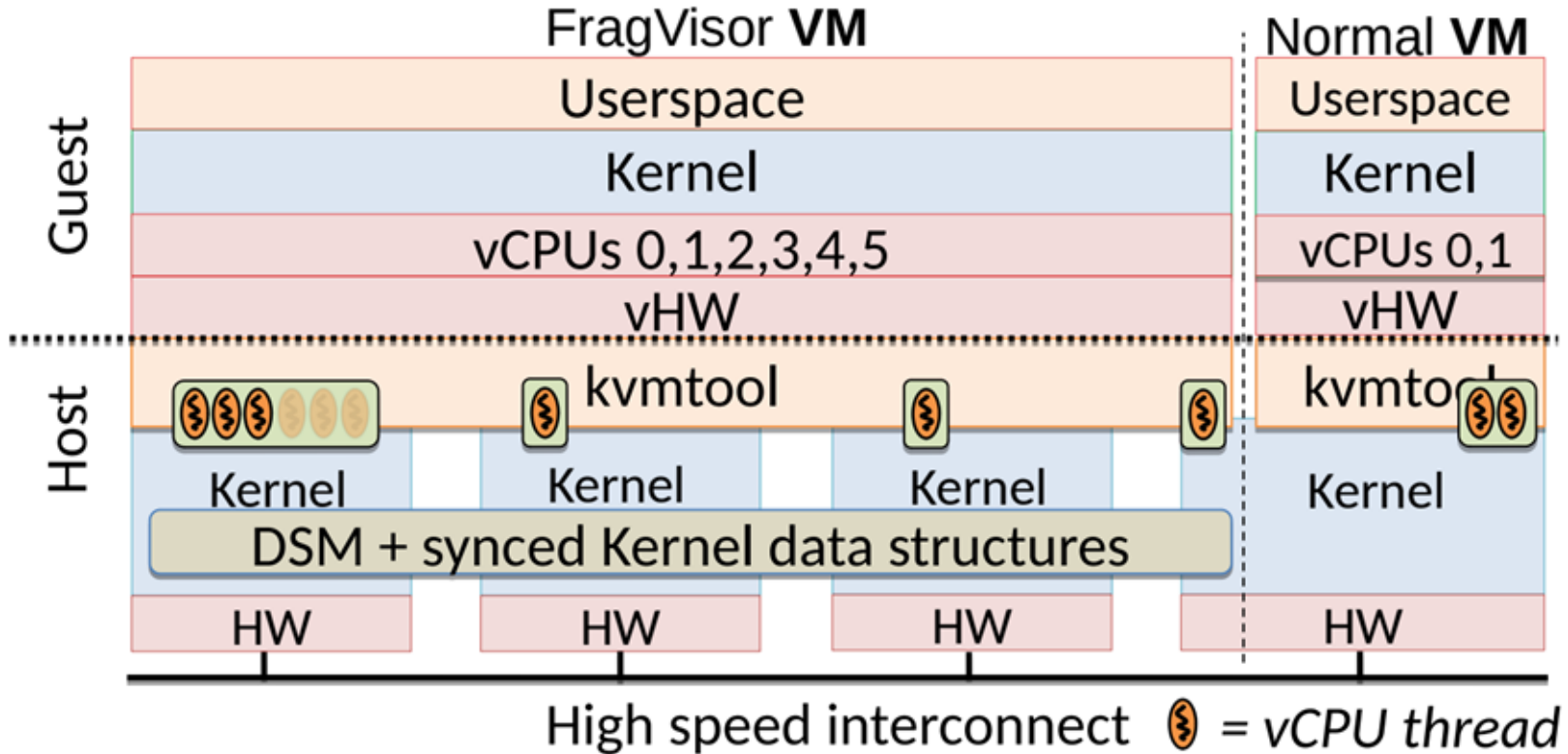
- **vRAM** mobility: DSM + synced data structures
- **vCPU** mobility: Runs vCPUs as threads in DSM => threads can migrate to different machines but they are kept consistent with DSM
- **I/O device** mobility: Device-CPU communication happens only with **memory mapped IO** => mapping exists in DSM (I/O Delegation)

All have further optimizations over basic DSM

Real performance and overheads will be determined in evaluation



Finally an Overview



Evaluation

- Can Aggregate VM be used to solve fragmentation?
- What are the eventual overheads (to motivate solution vs delayed-allocation, migration, etc.)
- How FragVisor, and its mechanisms, quantitatively differ from previous work.

Possible Overheads

With many VM slices on different machines:

- Communication needs to be frequent (ring buffer pair traffic).
 - Multiqueues on each vCPU/Hypervisor.
- High overhead from synchronization operations
 - DSM-bypass with paravirtualized network devices in each VM slice:
 1. TX/RX event occurs
 2. the network packet is piggybacked with the TX interrupt.
 3. interrupt sent to the VM slice that owns the physical network card
- Memory management is hard (Shadow PTEs in ETPs).
 - Possible hardware solutions in future (?)

Test Suite

- Multiple identical servers equipped with a Xeon E5-2620 v4 (2.1 GHz) and 32 GB of RAM.
- Servers are interconnected via an InfiniBand switch using Mellanox Connect-X4 adapters (56 Gbps).

Microbenchmarks are used to evaluate:

1. Cost of accessing remote memory and I/O
2. Overall performance of Aggregate VMs running on FragVisor in real-world scenarios
3. vCPU migration mechanism and modified version of Linux.
4. Overhead of distributed checkpointing mechanism

DSM Fault Traffic

The DSM overhead in FragVisor caused by EPT faults due to its consistency protocol.

3 Scenarios (All threads are executing R/W operations):

1. True sharing – all threads access the same location
2. False sharing – all threads access a different location but on the same page
3. No sharing

$O(N) \rightarrow$

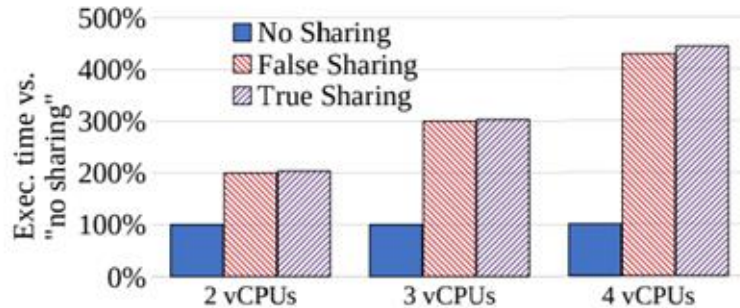


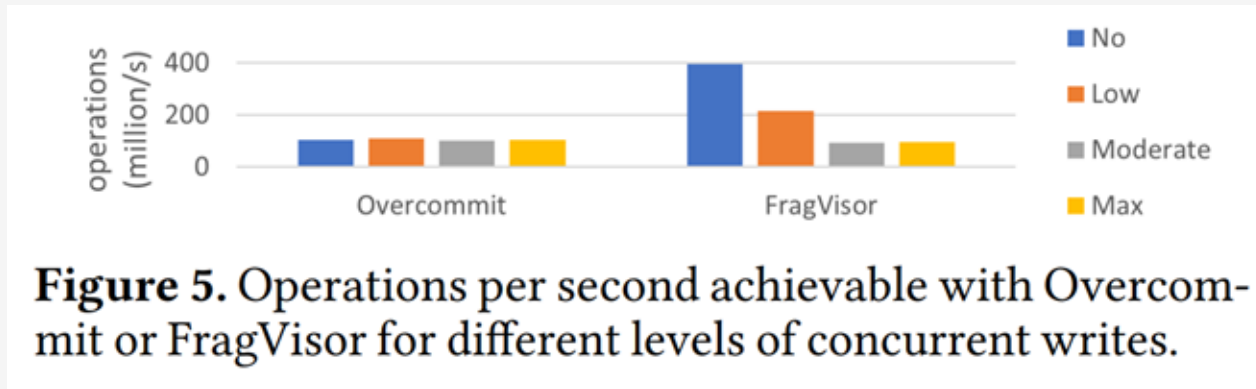
Figure 4. DSM overhead (EPT faults) by level of sharing.

DSM Concurrent Writes

Most intense overhead stemming from consistency protocol.

4 Scenarios (All threads are executing write operations): 1/2/3/4 vCPUs per page.

- Overcommitting means executing multiple vCPUs on a single pCPU.
- FragVisor enables vCPUs to run on different pCPUs
- Maximum total traffic achieved is 8 MB/s (64 Mbps), for Max-sharing.



Network I/O Delegation Overhead

Network throughput of a NGINX web server where the worker runs on a

- vCPU that is local to communicate with the client (local I/O)
- vCPU that is on a remote node (delegated I/O)

where the node sends 1000 varying-sized requests with 10 concurrent connections.

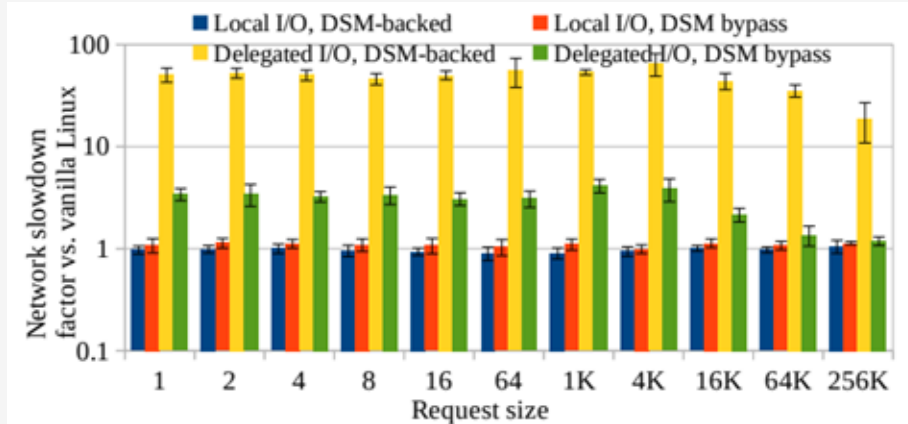
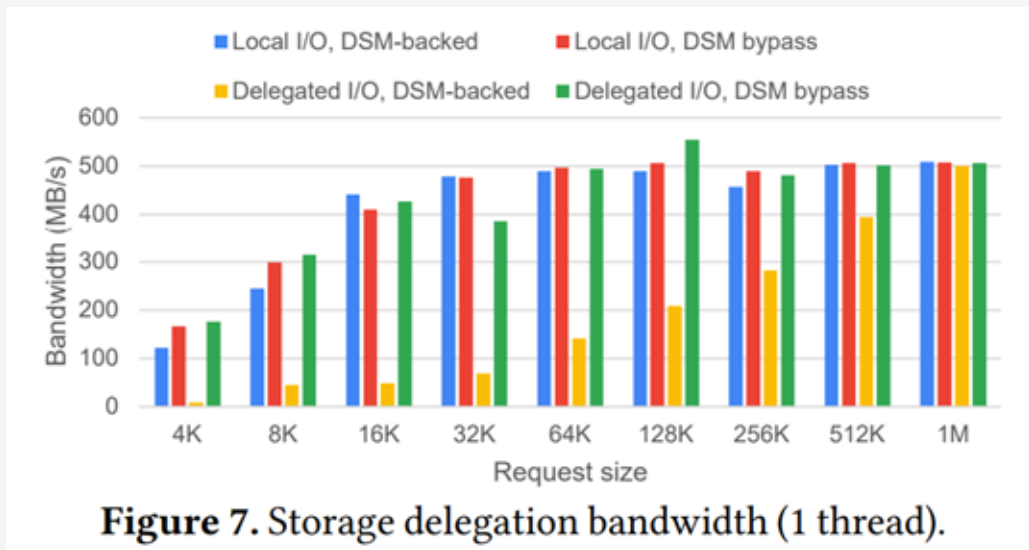


Figure 6. Network delegation overhead.

Storage I/O Delegation

The achievable bandwidth with one thread for local and remote (delegated) requests, with and without DSM, running on 1 thread.



Network Application: LEMP Stack

Opensource web stack consisting of NGINX, PHP, and MySQL.

- NGINX acts as a front-end HTTP server receiving requests from clients.
 - A worker thread is allocated in a local vCPU to avoid network delegation overheads demonstrated in previous experiments.
- NGINX invokes PHP which runs a worker thread on each vCPU except the one running the NGINX thread. (3 vCPU = 1 vCPU for NGINX + 2 vCPU for PHP)
- Setting the served page size to 2 MB, the average page size on the web.
- Processing time is varied from 25 to 500 ms for benchmark.
- 1000 requests with 10 concurrent connections.
- Compared overcommitment scenario and GiantVM (state-of-the-art).

Network Application: LEMP Stack



- In short processing times, FragVisor perform worse due to communication overhead.
- Speedup increases with computation time and the number of vCPUs.
- FragVisor is better at exploiting the parallelism brought by vCPUs.

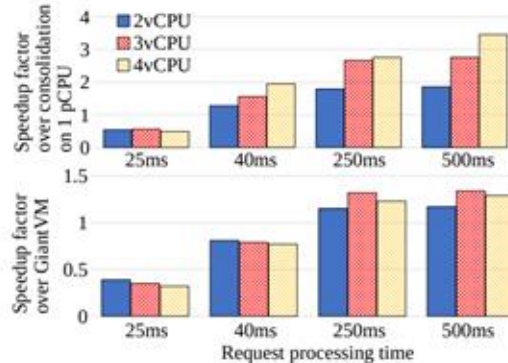


Figure 12. LEMP stack result for FragVisor, normalized to overcommitting (top) and GiantVM (bottom).



Serverless Computing on OpenLambda

1. Function as a Service(FaaS) Computing: Users upload and execute small pieces of code, or functions, in the cloud.
2. OpenLambda server is configured to spawn function on each available vCPU – on different VM slices upon receiving HTTP requests.
3. All vCPUs on 1 pCPU -> more processing using the same system resource.

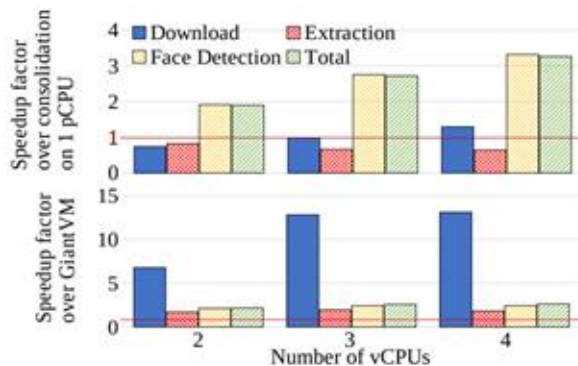


Figure 13. OpenLambda: FragVisor vs. overcommitting (top) and GiantVM (bottom).

Remarks and Conclusion

- Aggregate VM is efficient especially in higher workloads.
 - DSM-related overheads is standing against being universal for all workloads.
- FragVisor has been tested only on one hardware, and up to 8 nodes.
- Aggregate VMs can address resource fragmentation in data centers for certain types of workloads, without reducing HW resources or evicting VMs.
- FragVisor achieves higher throughput for exhaustive requests than the state-of-the-art GiantVM.
- With DSM bypass, FragVisor I/O delegation can achieve high performance.
 - It can offset the effects of distribution.
- FragVisor is faster than GiantVM in every phase of serverless computing.
- FragVisor shows a significant speedup over overcommitment in general.

References

- [1]<https://arxiv.org/ftp/arxiv/papers/1506/1506.07020.pdf>
- [2]<https://link.springer.com/article/10.1007/s00453-019-00643-z>
- [3]https://www.researchgate.net/publication/281790876_A_Provident_Resource_Defragmentation_Framework_for_Mobile_Cloud_Computing